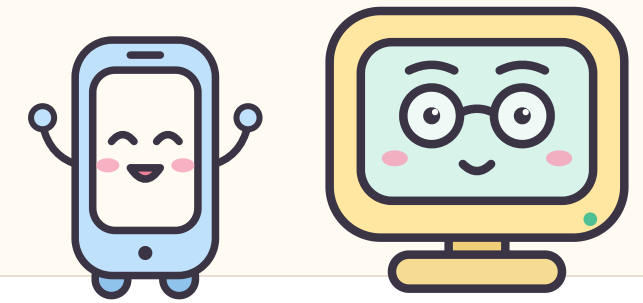




第 11 集 · 约 25 分钟

服务端④：连接管理

一个不需要密钥就能拖垮服务器的漏洞

11

从零写出 LanShare

这一集你会学到

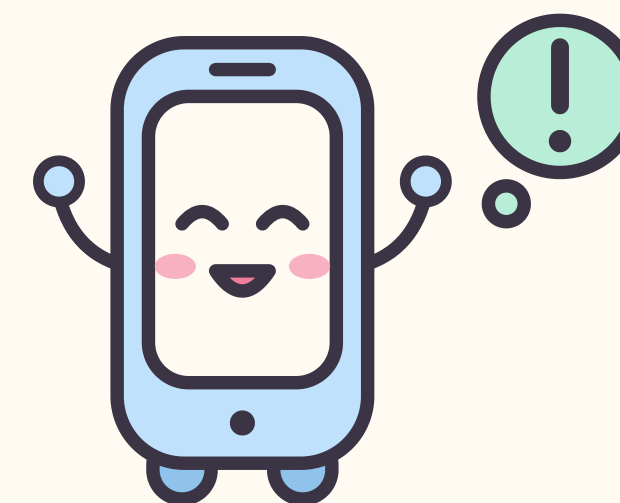
1. 为什么 axum 自带的 serve 不够：读请求头没有限
2. 名额和“票”：给类型实现 Drop，资源离开作用域自动归还
3. 异步的底层：poll、Pending、Waker 和 Pin
4. 自己写接收连接的循环：tokio::select!、tokio::spawn、给 hyper 配计时器



先想一个问题

所有数据接口都要密钥。那不知道密钥的人，能把服务器弄到无法服务吗？

能。在验证密钥之前，服务器得先收下连接、读请求头。只连上、不把请求头发完，连接就一直挂着。





只连不说话，连接就一直挂着



发生了什么

请求头没发完，服务器还在等

axum::serve 没给 hyper 配计时器

审查实测：300 个连接挂了 60 秒，一个都没断

多开几百个，每个都这样



修法：三件事一起上

- 给 hyper 配一个计时器，请求头 10 秒内必须收完

axum::serve 不给配，所以我们自己接收连接

- 再加一个“双向空闲”计时器，60 秒没有任何收发就断开

对付连上之后完全不说话、或者对方不收数据的连接

- 按来源 IP 限制连接数，本机回环单独一池

复验 N-1：全局上限会被一台机器占满，连本机界面都进不来



加依赖：hyper、hyper-util、tower

● ● ●

Cargo.toml

crates/lanshare/

第 10-29 行 • 共 29 行

```
10 path = "src/main.rs"
11
12 [dependencies]
13 axum = "0.8"
14 base64 = "0.22"
15 dirs = "6"
16 hex = "0.4"
17 hyper = { version = "1", features = ["http1", "server"] }
18 hyper-util = { version = "0.1.21", features = ["tokio", "server-graceful"] }
19 if-addr = "0.13"
20 lanshare-proto = { path = "../lanshare-proto" }
21 rand_core = { version = "0.6", features = ["getrandom"] }
22 serde = { version = "1", features = ["derive"] }
23 serde_json = "1"
24 tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
25 tower = { version = "0.5", features = ["util"] }
26 windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation", "Win32_Storage_FileSystem"] } 临时
27
28 [dev-dependencies]
29 tempfile = "3"
```



mod.rs: 声明 conn, 引入 Duration

mod.rs

crates/lanshare/src/server/

第 11-30 行 · 共 220 行

```

11 use std::time::Duration;
12
13 use axum::Router;
14 use axum::extract::{Request, State};
15 use axum::http::{HeaderValue, StatusCode, header};
16 use axum::middleware::{self, Next};
17 use axum::response::{IntoResponse, Response};
18 use axum::routing::get; 临时
19 use base64::Engine;
20 use rand_core::{OsRng, RngCore};
21
22 use crate::storage::Storage;
23
24 /// 只给本机主界面用的桌面操作（在资源管理器里定位文件、打开共享文件夹）。
25 pub trait Desktop: Send + Sync {
26     fn reveal(&self, path: &Path);
27     fn open_folder(&self, path: &Path);
28 }
29
30 pub type Logger = Arc<dyn Fn(&str) + Send + Sync>;

```



Config 和 AppState: 加上连接相关的字段

mod.rs

crates/lanshare/src/server/

第 114-133 行 • 共 243 行

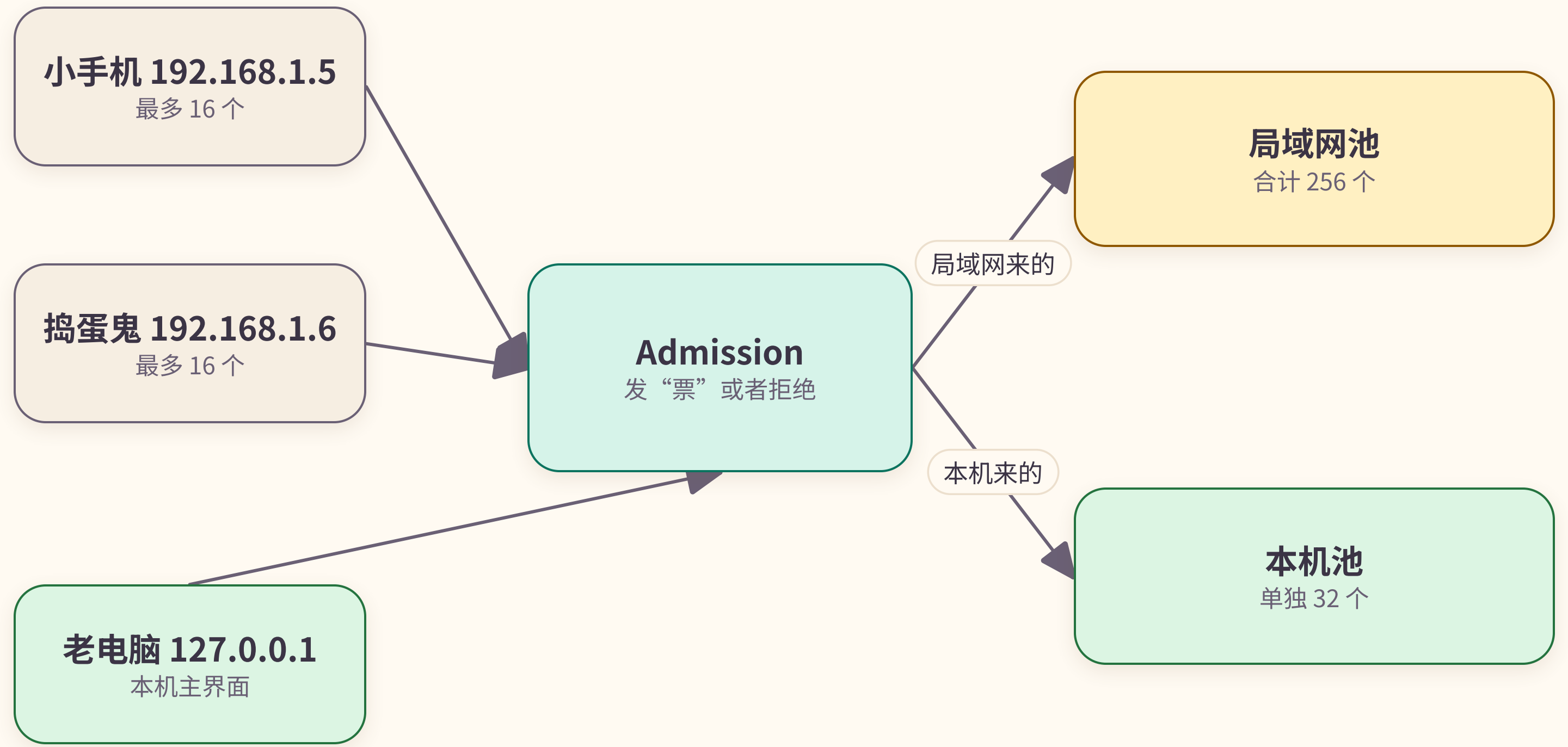
```

114         idle_timeout: config.idle_timeout,
115         max_connections: config.max_connections,
116         max_local_connections: config.max_local_connections,
117         max_connections_per_ip: config.max_connections_per_ip,
118         header_timeout: config.header_timeout,
119     }
120 }
121
122 pub fn key(&self) -> [u8; 32] {
123     self.key
124 }
125
126 pub fn server_id(&self) -> &str {
127     &self.server_id
128 }
129
130 pub fn pin(&self) -> String {
131     self.pin.lock().expect("口令锁不会中毒").clone()
132 }
133

```



连接准入：按 IP 计数，本机单独一池





conn.rs: 开头和计数

● ● ●

conn.rs

crates/lanshare/src/server/

第 16–35 行 • 共 35 行

```
16
17 use std::collections::HashMap;
18 use std::future::Future;
19 use std::io;
20 use std::net::IpAddr;
21 use std::pin::Pin;
22 use std::sync::{Arc, Mutex};
23 use std::task::{Context, Poll};
24 use std::time::Duration;
25
26 use tokio::io::{AsyncRead, AsyncWrite, ReadBuf};
27 use tokio::net::TcpStream;
28 use tokio::time::{Instant, Sleep};
29
30 #[derive(Default)]
31 struct Counts {
32     lan: usize,
33     local: usize,
34     per_ip: HashMap<IpAddr, usize>,
35 }
```



conn.rs: Admission, 发票的人

conn.rs crates/lanshare/src/server/

第 49-68 行 · 共 68 行

```

49     }
50
51     /// 放行就返回一张“票”，连接关闭（票被 drop）时自动退还名额；超过任何一个上限返回 None。
52     pub(crate) fn admit(&self, ip: IpAddr) -> Option<Ticket> {
53         let local = ip.is_loopback();
54         let mut counts = self.counts.lock().expect("锁不会中毒");
55         let mine = counts.per_ip.get(&ip).copied().unwrap_or(0);
56         let pool = if local { counts.local } else { counts.lan };
57         if mine >= self.per_ip || pool >= if local { self.max_local } else { self.max_lan } {
58             return None;
59         }
60         *counts.per_ip.entry(ip).or_default() += 1;
61         if local {
62             counts.local += 1;
63         } else {
64             counts.lan += 1;
65         }
66         Some(Ticket { counts: self.counts.clone(), ip, local })
67     }
68 }

```



用完自动归还：Python 和 Rust

Python

```
1 count += 1
2 try:
3     handle(conn)    # 处理连接
4 finally:
5     count -= 1      # 不管怎么退出，都要减回去
```

Rust

```
1 impl Drop for Ticket {
2     fn drop(&mut self) {
3         // 票离开作用域时，这里自动跑
4         // 把名额减回去
5     }
6 }
```



conn.rs: Ticket 和它的 Drop

● ● ● conn.rs crates/lanshare/src/server/

第 72-91 行 • 共 91 行

```
72     ip: IpAddr,
73     local: bool,
74 }
75
76 impl Drop for Ticket {
77     fn drop(&mut self) {
78         let mut counts = self.counts.lock().unwrap_or_else(|e| e.into_inner());
79         if self.local {
80             counts.local -= 1;
81         } else {
82             counts.lan -= 1;
83         }
84         if let Some(n) = counts.per_ip.get_mut(&self.ip) {
85             *n -= 1;
86             if *n == 0 {
87                 counts.per_ip.remove(&self.ip);
88             }
89         }
90     }
91 }
```



异步的底层：poll、Pending、Waker、Pin

接下来要亲手实现 AsyncRead，先认识四个词

● poll：运行时反复问一个 Future “好了没”

async/await 背后，就是不停地调用 poll

● Poll::Ready(值) / Poll::Pending

好了就给值；没好就先走开

● Waker：数据到了，叫醒运行时再来问一次

藏在 poll 的参数 cx 里

● Pin：保证这个值在内存里不会被挪动

现在只要知道 “Pin 包着的可变引用，就是一种特殊的 and mut self”



conn.rs: 带空闲计时的连接

● ● ● conn.rs crates/lanshare/src/server/

第 100–119 行 • 共 119 行

```
100
101 impl GuardedIo {
102     pub(crate) fn new(stream: TcpStream, ticket: Ticket, idle: Duration) -> Self {
103         GuardedIo { stream, _ticket: ticket, idle, deadline: Box::pin(tokio::time::sleep(idle)) }
104     }
105
106     fn touch(&mut self) {
107         let next = Instant::now() + self.idle;
108         self.deadline.as_mut().reset(next);
109     }
110
111     /// 底层还没数据可读/可写时, 看看是不是已经空闲太久了。
112     fn idle_expired(&mut self, cx: &mut Context<'_>) -> bool {
113         self.deadline.as_mut().poll(cx).is_ready()
114     }
115 }
116
117 fn timed_out() -> io::Error {
118     io::Error::new(io::ErrorKind::TimedOut, "连接空闲太久")
119 }
```



conn.rs: 实现 AsyncRead

conn.rs crates/lanshare/src/server/

第 116-135 行 · 共 135 行

```

116
117 fn timed_out() -> io::Error {
118     io::Error::new(io::ErrorKind::TimedOut, "连接空闲太久")
119 }
120
121 impl AsyncRead for GuardedIo {
122     fn poll_read(mut self: Pin<&mut Self>, cx: &mut Context<'_>, buf: &mut ReadBuf<'_>) -> Poll<io::Result<()>> {
123         let before = buf.filled().len();
124         match Pin::new(&mut self.stream).poll_read(cx, buf) {
125             Poll::Ready(result) => {
126                 if result.is_ok() && buf.filled().len() > before {
127                     self.touch();
128                 }
129                 Poll::Ready(result)
130             }
131             Poll::Pending if self.idle_expired(cx) => Poll::Ready(Err(timed_out())),
132             Poll::Pending => Poll::Pending,
133         }
134     }
135 }

```



conn.rs: 实现 AsyncWrite

conn.rs

crates/lanshare/src/server/

第 140–159 行 · 共 159 行

```

140         Poll::Ready(result) => {
141             if matches!(result, Ok(n) if n > 0) {
142                 self.touch();
143             }
144             Poll::Ready(result)
145         }
146         // 对方一直不收（慢读攻击）也算空闲
147         Poll::Pending if self.idle_expired(cx) => Poll::Ready(Err(timed_out())),
148         Poll::Pending => Poll::Pending,
149     }
150 }
151
152 fn poll_flush(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<io::Result<()>> {
153     Pin::new(&mut self.stream).poll_flush(cx)
154 }
155
156 fn poll_shutdown(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<io::Result<()>> {
157     Pin::new(&mut self.stream).poll_shutdown(cx)
158 }
159 }

```



conn.rs: 测试准入逻辑

conn.rs crates/lanshare/src/server/

第 162-181 行 • 共 181 行

```
162 mod tests {
163     use super::*;
164
165     #[test]
166     fn admission_counts_per_ip_and_per_pool() {
167         let a = Admission::new(3, 1, 2);
168         let lan1: IpAddr = "192.168.1.5".parse().unwrap();
169         let lan2: IpAddr = "192.168.1.6".parse().unwrap();
170         let local: IpAddr = "127.0.0.1".parse().unwrap();
171         let t1 = a.admit(lan1).unwrap();
172         let _t2 = a.admit(lan1).unwrap();
173         assert!(a.admit(lan1).is_none(), "同一 IP 最多 2 个");
174         let _t3 = a.admit(lan2).unwrap();
175         assert!(a.admit(lan2).is_none(), "局域网池子满了 (3 个)");
176         let _l = a.admit(local).unwrap();
177         assert!(a.admit("127.0.0.2".parse().unwrap()).is_none(), "本机池子 1 个");
178         drop(t1);
179         assert!(a.admit(lan1).is_some(), "票退还后名额回来");
180     }
181 }
```



mod.rs: serve, 自己接收连接 (上)

mod.rs crates/lanshare/src/server/

第 246–265 行 • 共 265 行

```
246
247 /// 同上, `shutdown` 完成时停止接受新连接, 等进行中的请求结束 (最多 3 秒; 托盘“退出”时用)。
248 ///
249 /// 没用 `axum::serve`: 它不给 hyper 配计时器, 读请求头没有时限 (独立安全审查 I-3、复验 N-1)。
250 /// 这里自己接收连接: 先过准入 (每 IP 上限、本机单独一池), 再交给 hyper, 配上计时器和请求头时限。
251 pub async fn serve_with_shutdown(
252     listener: tokio::net::TcpListener,
253     state: Arc<AppState>,
254     shutdown: impl std::future::Future<Output = ()> + Send + 'static,
255 ) -> std::io::Result<()> {
256     use axum::extract::ConnectInfo;
257     use hyper_util::rt::{TokioIo, TokioTimer};
258     use tower::ServiceExt;
259
260     let app = router(state.clone());
261     let admission = conn::Admission::new(state.max_connections, state.max_local_connections, state.max_connections_per_ip);
262     let graceful = hyper_util::server::graceful::GracefulShutdown::new();
263     let mut shutdown = std::pin::pin!(shutdown);
264     /// 供 handler 使用的 State 类型别名。
265     pub(crate) type St = State<Arc<AppState>>;
```



mod.rs: serve, 接收循环 (中)

mod.rs

crates/lanshare/src/server/

第 265-284 行 · 共 284 行

```

265         let accepted = tokio::select! {
266             accepted = listener.accept() => accepted,
267             _ = &mut shutdown => break,
268         };
269         let (stream, peer) = match accepted {
270             Ok(pair) => pair,
271             // 对方刚连上就断了之类的连接级错误: 直接接下一个 (和 axum 自带的处理一样, 第三轮复验 R3-3)
272             Err(e) if is_connection_error(&e) => continue,
273             // 句柄耗尽之类的错误: 稍等再接; 等的时候也响应“退出”
274             Err(_) => tokio::select! {
275                 _ = tokio::time::sleep(Duration::from_millis(50)) => continue,
276                 _ = &mut shutdown => break,
277             },
278         };
279         // 超过上限的连接直接关掉 (stream 在这里被 drop), 不排队
280         let Some(ticket) = admission.admit(peer.ip()) else { continue };
281         let _ = stream.set_nodelay(true);
282         let io = TokioIo::new(conn::GuardedIo::new(stream, ticket, state.idle_timeout));
283     /// 供 handler 使用的 State 类型别名。
284     pub(crate) type St = State<Arc<AppState>>;
    
```



mod.rs: serve, 交给 hyper (下)

mod.rs

crates/lanshare/src/server/

第 293-312 行 · 共 312 行

```

293         .max_buf_size(64 * 1024); // 本协议的请求头都很小; 正文是流式读取的, 不受这个缓冲限制
294     let connection = graceful.watch(builder.serve_connection(io, service));
295     tokio::spawn(async move {
296         let _ = connection.await;
297     });
298 }
299 tokio::select! {
300     _ = graceful.shutdown() => {}
301     _ = tokio::time::sleep(Duration::from_secs(3)) => {}
302 }
303 Ok(())
304 }
305
306 fn is_connection_error(e: &std::io::Error) -> bool {
307     use std::io::ErrorKind::{ConnectionAborted, ConnectionRefused, ConnectionReset};
308     matches!(e.kind(), ConnectionAborted | ConnectionRefused | ConnectionReset)
309 }
310
311 /// 供 handler 使用的 State 类型别名。
312 pub(crate) type St = State<Arc<AppState>>;

```



让 demo 用上我们自己的 serve

● ● ●

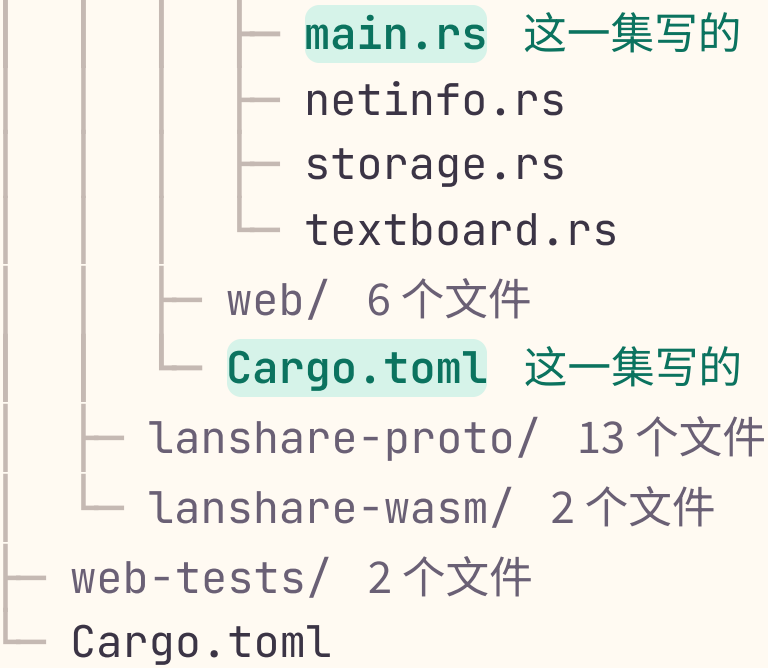
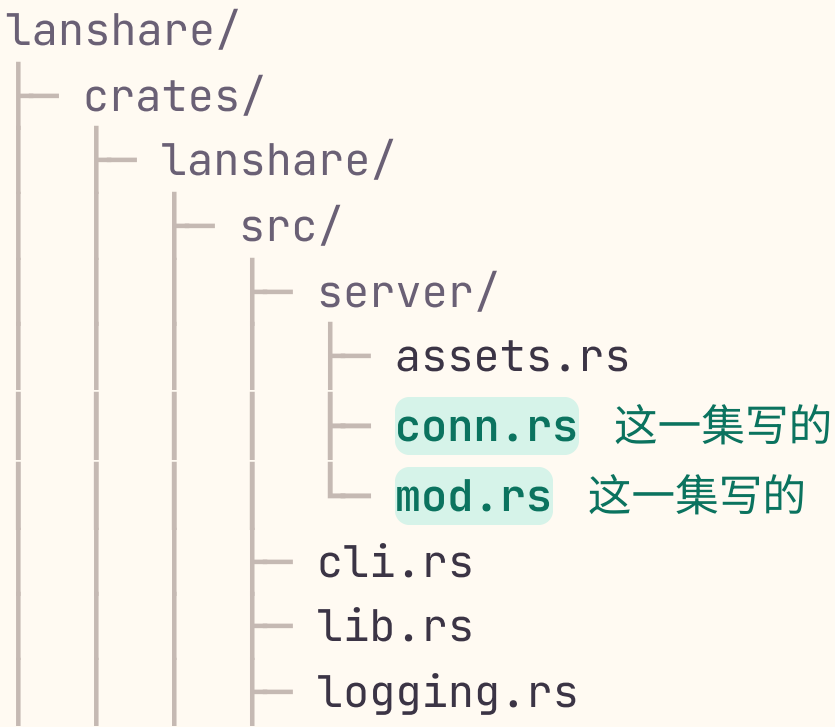
main.rs crates/lanshare/src/ 临时

第 1-12 行 • 共 12 行

```
1 fn main() {
2     println!("LanShare 骨架跑起来了, 协议版本: {}", lanshare_proto::PROTOCOL);
3     // 临时: 换成我们自己写的 serve, 真正的入口在第 16 集 临时
4     let config = lanshare::server::Config::new(std::env::temp_dir().join("lanshare-demo")); 临时
5     let state = std::sync::Arc::new(lanshare::server::AppState::new(config, 8765)); 临时
6     println!("主界面地址: {}", state.local_entry_url()); 临时
7     let runtime = tokio::runtime::Runtime::new().unwrap(); 临时
8     runtime.block_on(async { 临时
9         let listener = tokio::net::TcpListener::bind("127.0.0.1:8765").await.unwrap(); 临时
10        lanshare::server::serve(listener, state).await.unwrap(); 临时
11    }); 临时
12 }
```



连接管理写好了





编译、跑测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 6.20s

PS D:\code\lanshare> cargo test -p lanshare --lib
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 3.22s
  Running unittests src\lib.rs (target\debug\deps\lanshare-1c33ee2213d0c6f1.exe)

running 23 tests
..... (省略 7 行, 前面几集的测试, ok)
test server::conn::tests::admission_counts_per_ip_and_per_pool ... ok
..... (省略 15 行, 前面几集的测试, ok)

test result: ok. 23 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.02s
```





换上自己的 serve，再跑一次

```
Windows PowerShell

PS D:\code\lanshare> cargo run -p lanshare
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.30s
    Running `target\debug\LanShare.exe`
LanShare 骨架跑起来了，协议版本: lanshare/2
主界面地址: http://127.0.0.1:8765/#DKbdNrvmw-Qn8ZMUXIuLp85MyZ2TSz-YsNrSnjllV0

PS D:\code\lanshare> curl.exe -si http://127.0.0.1:8765/api/server
HTTP/1.1 200 OK
content-type: application/json
content-security-policy: default-src 'none'; script-src 'self' 'wasm-unsafe-eval'; style-src 'self'; img-src 'self'
blob:; connect-src 'self'; base-uri 'none'; form-action 'none'; frame-ancestors 'none'
```





选择题

Ticket 实现了 Drop。它的 drop 里把名额减回去。什么时候会被调用？

- A 票离开作用域、被丢弃时，编译器自动调用
- B 每次有新连接进来时
- C 要手动调用 ticket.drop()



选择题

Ticket 实现了 Drop。它的 drop 里把名额减回去。什么时候会被调用？

A 票离开作用域、被丢弃时，编译器自动调用

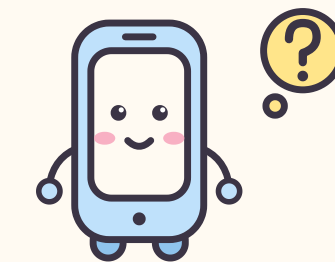
一个值离开作用域就会被丢弃，Drop 的 drop 自动跑。连接处理完、GuardedIo 被丢弃时，里面的票跟着被丢弃，名额就退回去了。你从不手动写退还名额的代码。

B 每次有新连接进来时

新连接触发的是 admit，不是 drop。

C 要手动调用 ticket.drop()

drop 由编译器自动插入，手动调用 .drop() 反而是不允许的。



选择题

poll_read 里，底层返回 Poll::Pending。为什么还要调用一次 self.idle_expired(cx)?

- A 为了把数据读出来
- B 为了判断是不是空闲太久，顺便把 Waker 交给计时器，到点能被叫醒
- C 为了让 CPU 空转等待



选择题

poll_read 里，底层返回 Poll::Pending。为什么还要调用一次 self.idle_expired(cx)?

A 为了把数据读出来

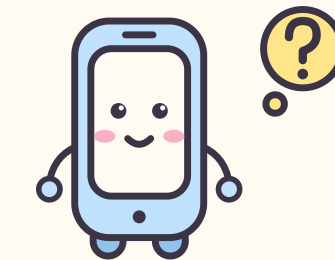
Pending 恰恰表示现在没有数据可读。

C 为了让 CPU 空转等待

异步靠 Waker 叫醒，不空转；空转正是它要避免的。

B 为了判断是不是空闲太久，顺便把 Waker 交给计时器，到点能被叫醒

没数据时正是检查空闲超时的时机。idle_expired 里 poll 了计时器，把 cx 里的 Waker 登记给它，这样即使一直没有数据，到点时计时器也会叫醒运行时来关掉连接。



选择题

第一版的连接上限是“全局 128 个，不分来源”。复验为什么说它让攻击门槛更低了？

A 128 太多了

B 一台机器就能占满全局名额，连本机的主界面都进不来

C 128 个连接会耗尽内存



选择题

第一版的连接上限是“全局 128 个，不分来源”。复验为什么说它让攻击门槛更低了？

A 128 太多了

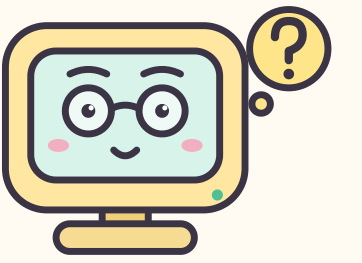
问题不在数量，在“不分来源”。

C 128 个连接会耗尽内存

连接本身占的内存很小，问题是名额被独占。

B 一台机器就能占满全局名额，连本机的主界面都进不来

名额不分来源，局域网里一台机器隔一会儿发一个字节占住连接，就把全部名额吃光，谁都连不上，包括电脑自己那个“重新开启口令登录”的界面。改成按 IP 计数、本机单独一池，才解决。



请求头时限交给了 hyper（10 秒），为什么还要我们自己再做一个 60 秒的空闲计时器？两者各管什么？



想一想

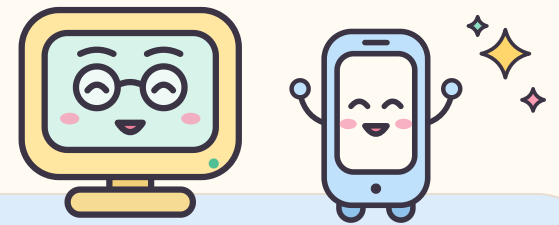
请求头时限交给了 hyper（10 秒），为什么还要我们自己再做一个 60 秒的空闲计时器？两者各管什么？

hyper 的请求头时限管的是“等请求头”：一个请求的请求头 10 秒内要收完，keep-alive 连接空着等下一个请求超过 10 秒也会被关。可请求头收完之后呢？一个连接正常传大文件，可能要好几分钟，不能一刀切。我们的空闲计时器管的是“任何方向 60 秒内有没有收发”：只要还在传，就一直续命；一旦卡住不动，60 秒就断。它还能对付对方不收数据的慢读，那是 hyper 的请求头时限管不到的。两者一个管开头，一个管全程。



这一集写了什么

- ✓ axum 自带的 serve 不给 hyper 配计时器，只连不说话的连接能一直挂着（审查 I-3）
- ✓ Admission 按 IP 计数、本机单独一池；Ticket 实现 Drop，连接关闭自动退还名额
- ✓ 第一次掀开 async 的盖子：poll 返回 Ready 或 Pending，Waker 负责叫醒，Pin 保证不被挪动
- ✓ 自己写接收循环：select! 同时等连接和退出，spawn 一个连接一个任务，给 hyper 配请求头时限
- ✓ 服务端的骨架到此齐了：状态、路由、中间件、连接管理



下一集

全课完结，感谢收看